

MongoDB Lab Manual



**RAJALAKSHMI
ENGINEERING COLLEGE**

An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai



**Department of Information
Technology**

Laboratory Record Regulation - 2023

MongoDB Essentials

IT23431



RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institution, Affiliated to Anna University Rajalakshmi Nagar,
Thandalam - 602 105



DEPARTMENT OF INFORMATION TECHNOLOGY

IT23431 - MONGODB ESSENTIALS

(Regulation 2023)

LAB MANUAL

Name :

Register No. :

Year / Branch / Section :

Semester :

Academic Year :

Course Code	Course Title (Laboratory Integrated Theory Course)	L	T	P	C
IT23431	Mongo DB Essentials	2	0	2	3

LIST OF EXPERIMENTS			
SNO	NAME OF THE EXPERIMENT	PAGE NO	SIGN
1	Mongo DB installation and basic set up		
2	Creating and dropping tables with various constraints		
3	Performing CRUD Operations		
4	Applying CRUD Operations		
5	Applying filtering and projections		
6	Perform Schema Design and Indexing		
7	Building aggregation pipelines		
8	Performing Data Transformation		
9	Establishing connection between Mongo Db and java		
10	Develop a CRUD Application using Mongo Db(Mini Project)		

Ex. Nos. 1	MONGO DB INSTALLATION AND BASIC SET UP
Date : 12.01.2026	

AIM

To install Mongo DB and do the basic set up

PROCEDURE

1. System Requirements and Pre-installation Checklist

Supported Windows Versions: Windows Server 2022, Windows Server 2019, Windows 11 (64-bit, x86_64).

WSL Not Supported: MongoDB does not support Windows Subsystem for Linux (WSL).

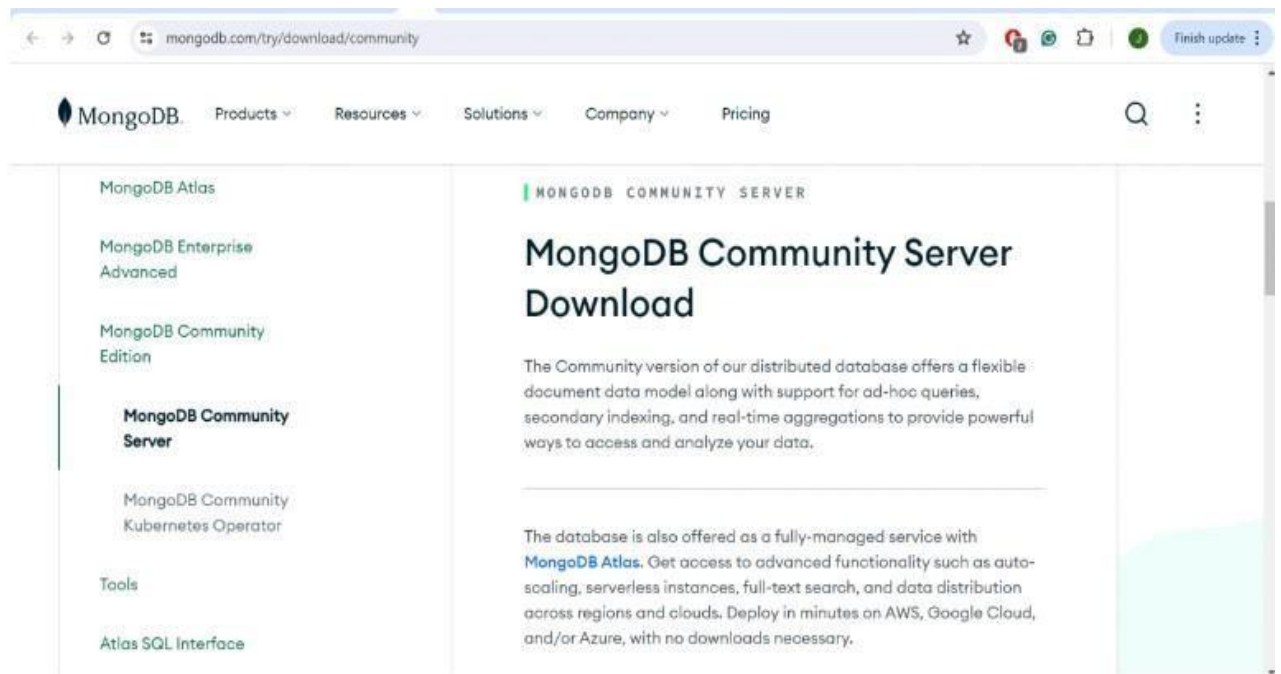
Disk Space: Minimum 10 GB of free space (additional space for data).

Hardware Requirements:

- RAM: Minimum 4 GB.
- CPU: Intel (Sandy Bridge or later), AMD (Bulldozer or later).

2. Downloading and Setting Up MongoDB

Go to the MongoDB official download page to access the MongoDB Community Server section and begin downloading the installation package for Windows.

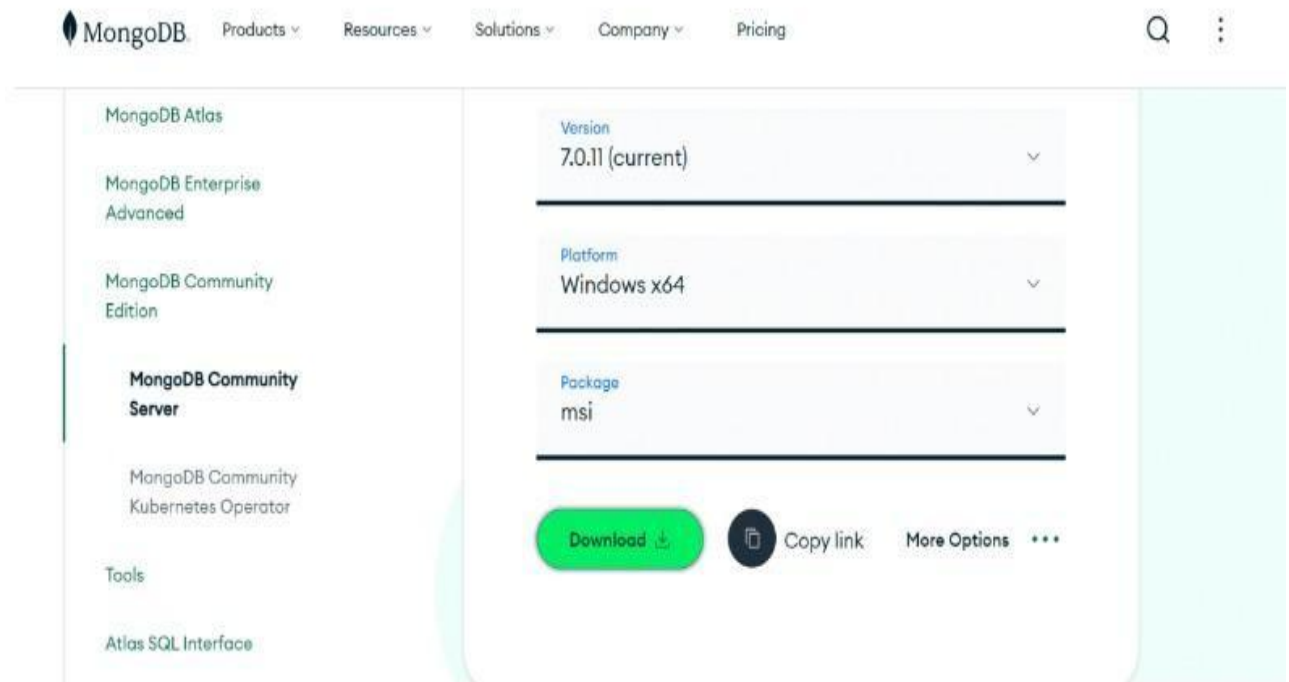


3. Selecting the Correct MongoDB Version and downloading the MongoDB Installer (MSI Package)

- ❖ Choose the latest stable release for optimal performance and security, unless you need an older version.
- ❖ Ensure you select the 64-bit version of MongoDB, as 32-bit versions are not compatible with MongoDB.

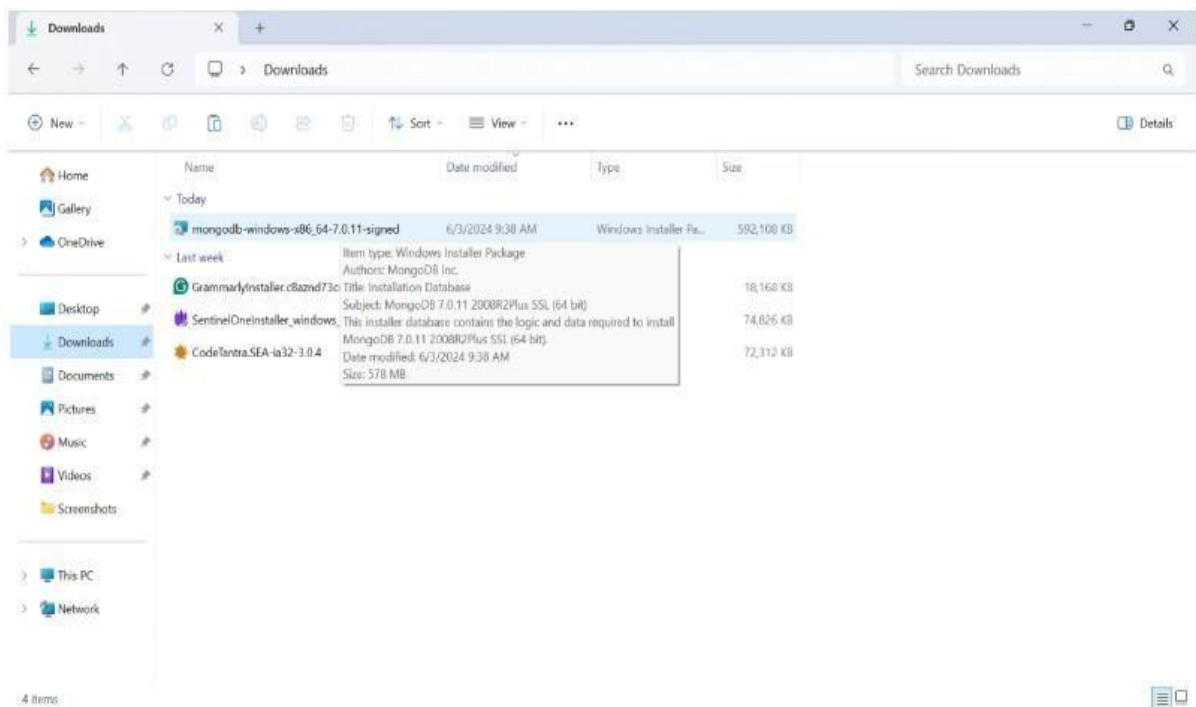
Downloading:

- ❖ Select the MSI package type for Windows and click Download.
- ❖ The .msi installer file will download to your default location (usually the Downloads folder).
- ❖ Double-click the .msi file to launch the installation wizard and begin setup.

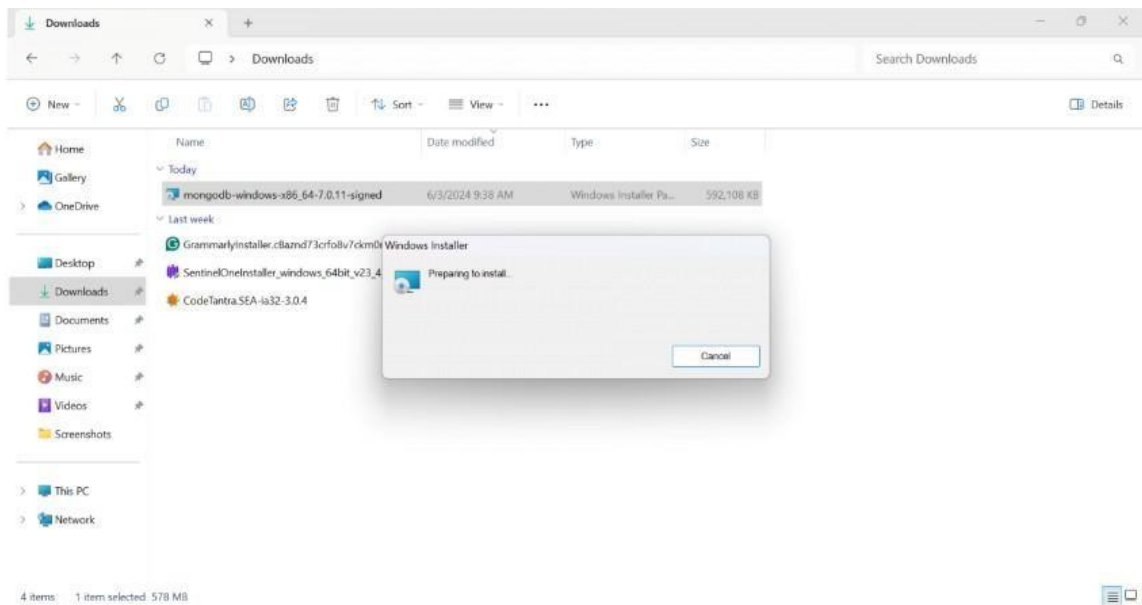


4. Installing MongoDB on Windows

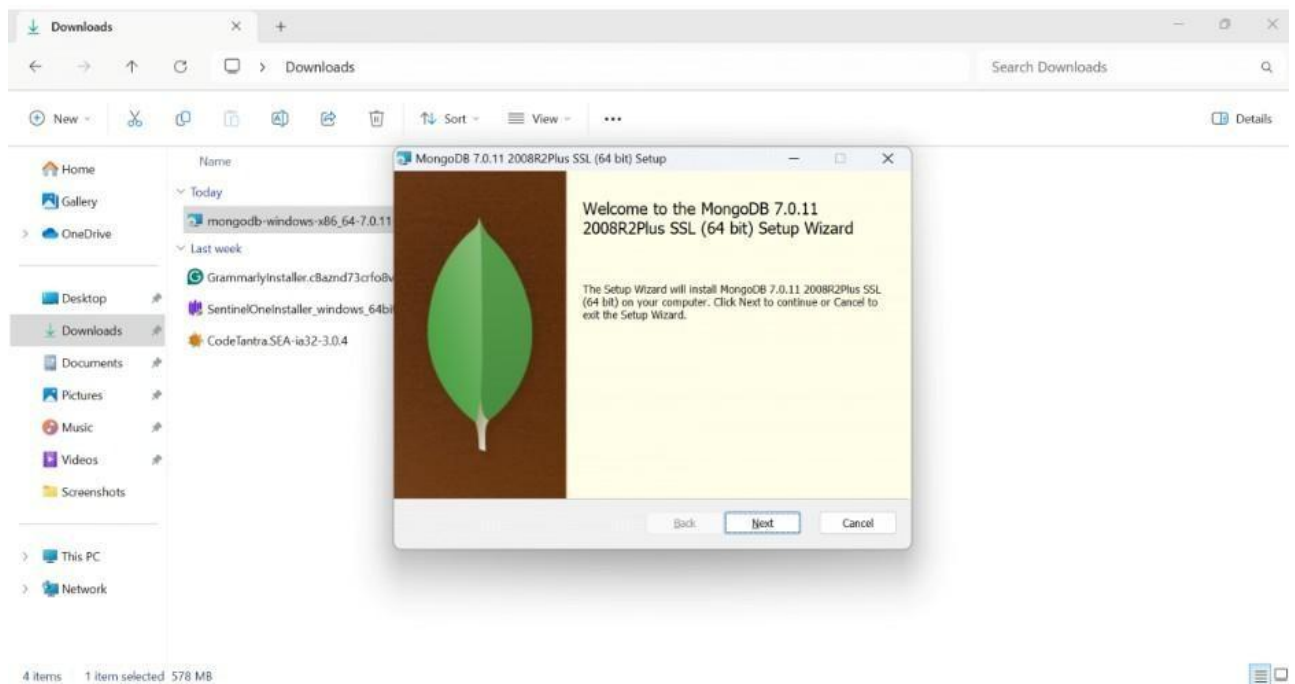
- i. Locate the downloaded .msi file in the Downloads folder.

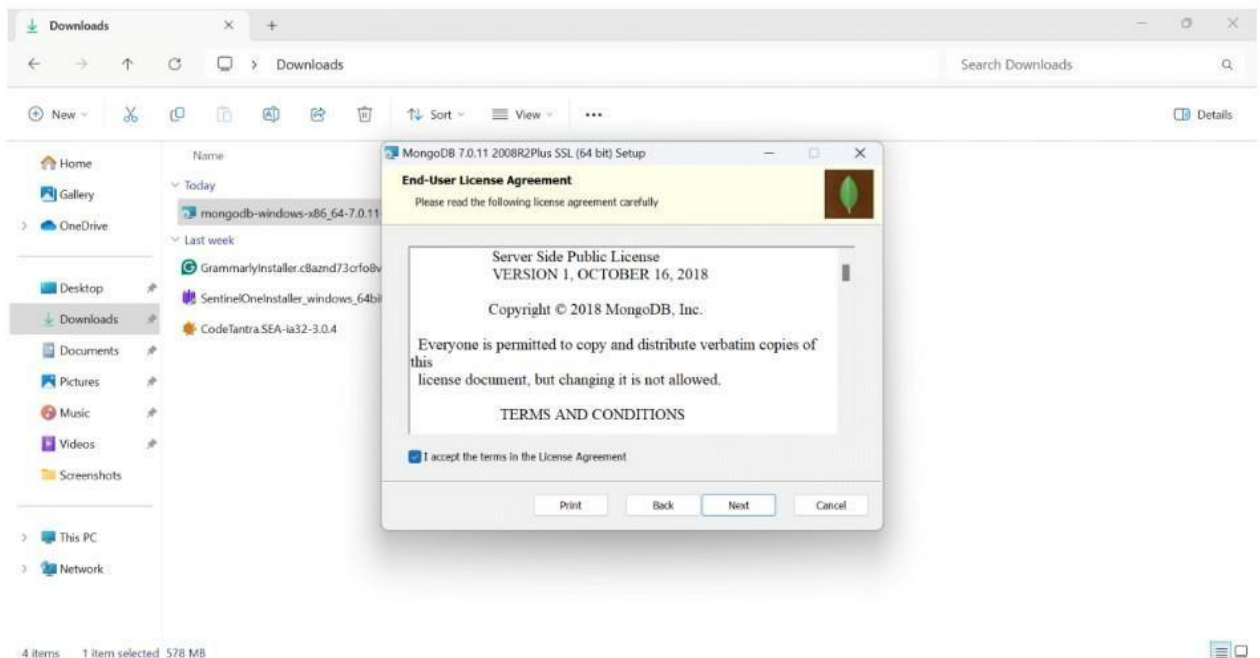


- ii. Double-click the file to start the installation wizard.



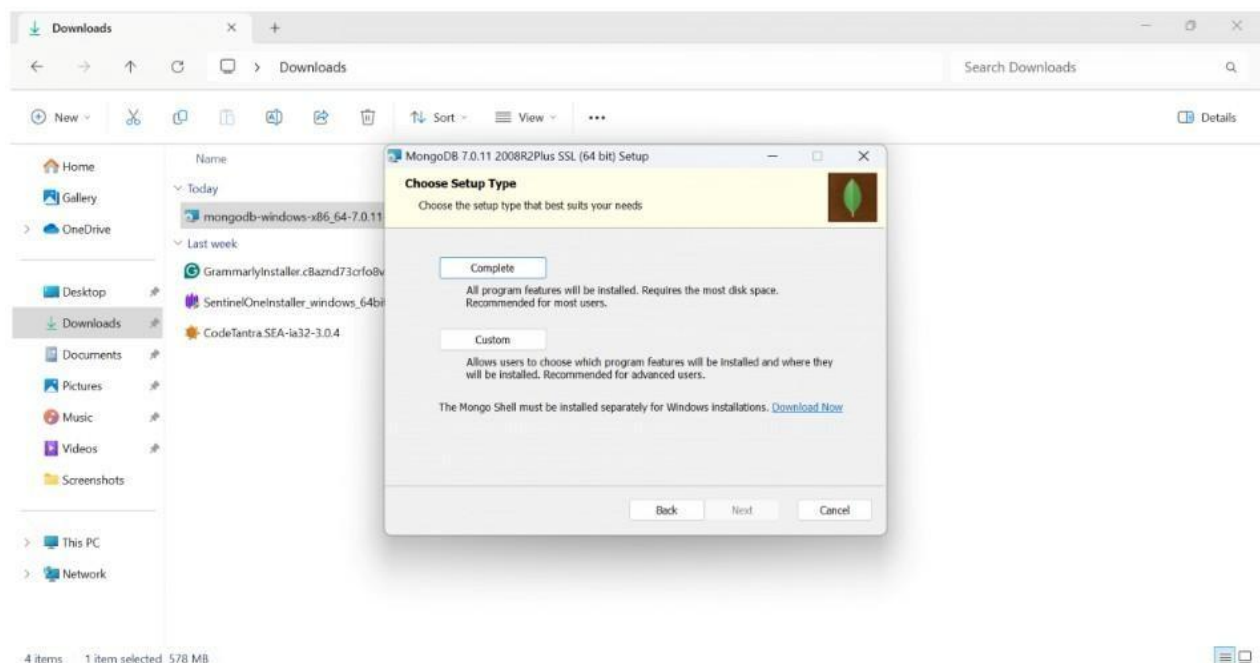
iii. Follow the prompts in the wizard to complete the setup.





5. Choosing the Installation Type

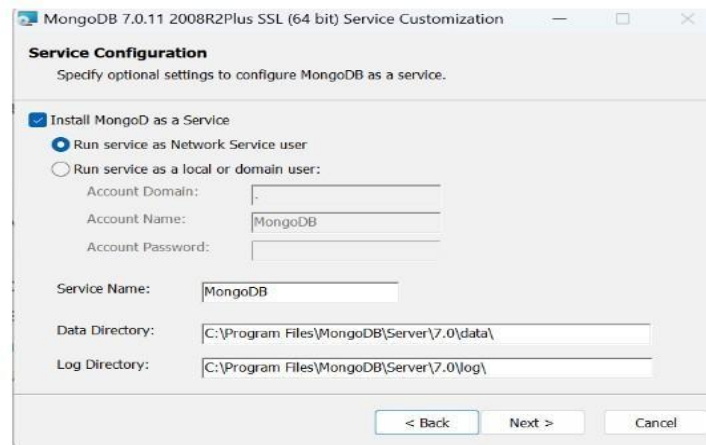
- i. Choose Complete Setup for a default installation with all features (recommended).
- ii. Select Custom Setup to choose specific components and installation locations.



6. Configuring Network Options and Data Directories:

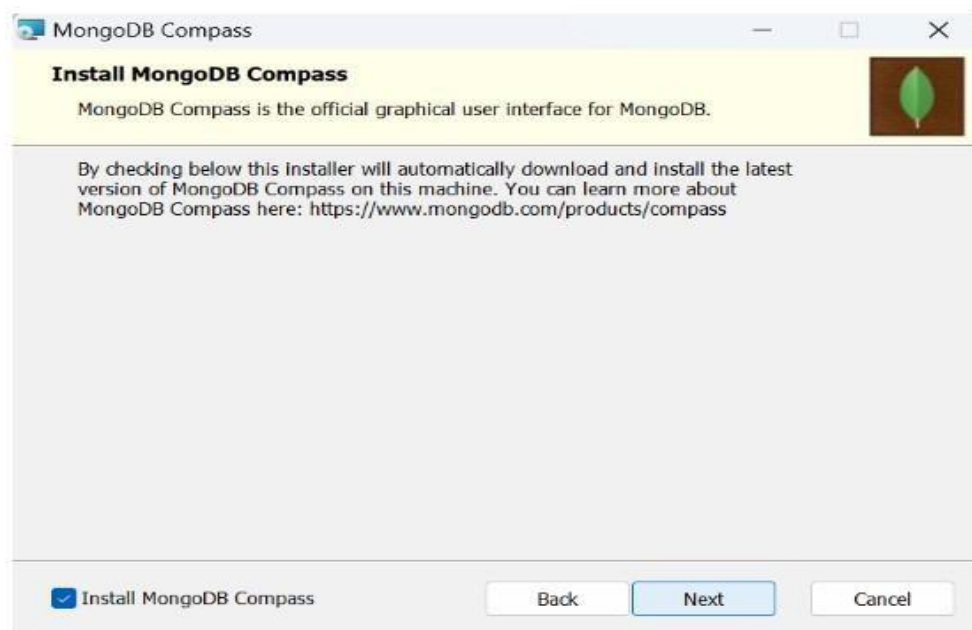
Service & Network Setup: Choose to run MongoDB as a service and configure network settings (localhost or custom IP).

Install & Launch: Complete installation, then start MongoDB with mongod and verify it's running with the "waiting for connections" message.

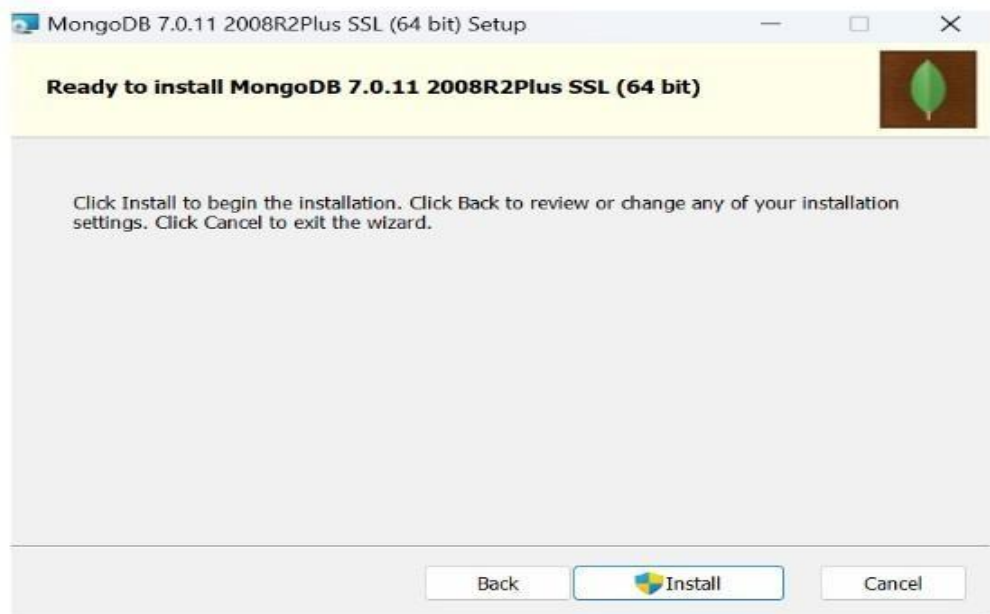


7. Completing the Installation Process

- i. **Install MongoDB Compass (Optional):** You can choose to install MongoDB Compass along with the MongoDB server. This GUI tool helps in managing your database.



- ii. **Finalize Installation:** Review your settings and click 'Install' to begin the installation process. A progress bar will display the installation status.



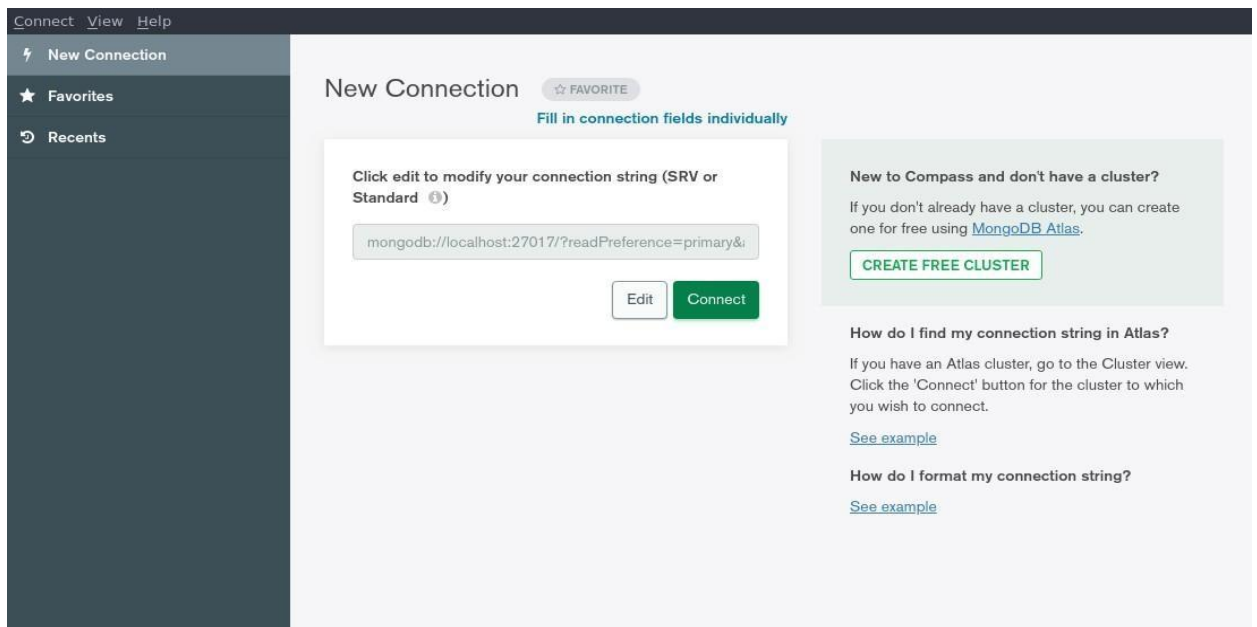
- iii. **Launch MongoDB:** Once the installation is complete, you can start MongoDB using the `mongod` command in the command prompt if it's not set to run as a service. Ensure that the `mongod.exe` process is running successfully by checking for the 'waiting for connections' message in the console output.



MongoDB Compass is being installed.

It will launch once it is done.

- iv. **Mongo DB Start Screen:**



RESULT:

MongoDB compass is successfully installed.

Ex. Nos. 2	CREATING AND DROPPING TABLES WITH VARIOUS CONSTRAINTS
Date: 19.01.2026	

CREATE

Creating a database using the Use <db> command:

The use <db> command selects a specific database for executing queries in a MongoDB session. If the specified database does not exist, MongoDB creates it when data is inserted.

syntax : **use <databaseName>** Query:

use employee

output:

```
> use employee
< switched to db employee
```

Creating a collection using the `db.createCollection("<collectionName>")` command :

The `db.createCollection("<collectionName>")` command in MongoDB creates a new collection with the specified name in the current database.

syntax : `db.createCollection("<collectionName>")` **Query:**

```
db.createCollection("employees")
```

output:

```
> db.createCollection("employees")
< { ok: 1 }
```

Inserting Data into a collection using `db.<collectionName>.insertOne({<document>})` :

The `db.<collectionName>.insertOne({<document>})` command in MongoDB inserts a new document into the specified collection. .

syntax : `db.<collectionName>.insertOne({<document>})`

Query: `db.employees.insertOne({name:"nick",age:25,desig:"HR",
salary:30000});`

output:

```
> db.employees.insertOne({name:"nick",age:25,desig:"HR",salary:30000});  
< {  
  acknowledged: true,  
  insertedId: ObjectId('67a84015fa341e0916c7ebea')  
}
```

Viewing all collections in a database using `show collections` command:

The `show collections` command in MongoDB lists all collections in the currently selected database.

syntax : `show collections` Query:

`show collections`

output:

```
> show collections  
< emp  
  employees
```

Checking if a collection exists using the

`db.getCollectionNames().includes("<collectionName>")` command:

The `db.getCollectionNames().includes("<collectionName>")` command in MongoDB checks if a specific collection exists in the current database and returns true or false.

syntax : `db.getCollectionNames().includes("<collectionName>")`

Query: `db.getCollectionNames().includes("employees")`

output:

```
> db.getCollectionNames().includes("employees")  
< true
```

DROP

Dropping a collection using `db.<collectionName>.drop()` the command:

The `db.<collectionName>.drop()` command in MongoDB permanently deletes the specified collection and all its documents from the database

syntax : `db.<collectionName>.drop()` **Query:**

`db.employees.drop()`

output:

```
> db.employees.drop()
< true
```

Verifying if a collection has been dropped using show collections command:

The show collections command in MongoDB lists all collections in the currently selected database.

syntax : `show collections` **Query:**

`show collections`

output:

```
> show collections
< emp
```

Testing automatic collection creation using the

`db.<newCollectionName>.insert({<document>})` command:

The `db.<newCollectionName>.insert({<document>})` command in MongoDB inserts a new document into the specified collection, creating the collection if it does not already exist.

syntax : `db.<newCollectionName>.insert({<document>})`

Query: `db.student.insertOne({name:"Riley",age:17,marks:95});`

output:

```
> db.student.insertOne({name:"Riley",age:17,marks:95});
< {
  acknowledged: true,
  insertedId: ObjectId('67a84761fa341e0916c7ebee')
}
```

DROP DATABASE

Dropping a database using the `db.dropDatabase()` command:

The `db.dropDatabase()` command is used to permanently delete the selected database along with all its collections and documents. Before executing, ensure you have switched to the correct database using the `use database_name` command.

syntax : `db.dropDatabase()` **Query:**

`db.dropDatabase()`

output:

```
> use employee
< switched to db employee
> db.dropDatabase()
< { ok: 1, dropped: 'employee' }
```

RESULT:

Creation and Dropping of tables with constraints is executed in MongoDB successfully.

Ex. Nos. 3	PERFORMING CRUD OPERATIONS
Date: 02.02.2026	

Creating a Collection

- Command: `db.createCollection("<collectionName>")`

4. Inserting Data (Create Operation) `Insert`

One Document

Insertion with Nested Fields Insert

Multiple Documents Insertion with

Specified `_id` Insertion with

Arrays

5. Reading Data (Read Operation)

- Finding All Documents
 - Command: `db.<collectionName>.find()`
- Finding Specific Documents
 - Command: `db.<collectionName>.find({<query>})`
- Finding a Single Document
 - Command: `db.<collectionName>.findOne({<query>})`

6. Updating Data (Update Operation) • Updating

a Single Document

- Command: `db.<collectionName>.updateOne({<query>}, {$set: {<update>}})`
- Updating Multiple Documents
 - Command: `db.<collectionName>.updateMany({<query>}, {$set: {<update>}})`

7. Deleting Data (Delete Operation)

- Deleting a Single Document
 - Command: `db.<collectionName>.deleteOne({<query>})`
- Deleting Multiple Documents
 - Command: `db.<collectionName>.deleteMany({<query>})`

RESULT :

CRUD Operations are performed successfully in Mongoddb

Ex. Nos. 4	APPLYING CRUD OPERATIONS
Date: 09.02.2026	

SCENARIO 1: PROJECT TASK MANAGEMENT

The company assigns tasks to employees for various projects. Each task includes the task name, assigned employee, status, and deadline.

1. Create Database and Collections:

Create a MongoDB database called **taskdb**. In this database, create a collection named **tasks** to store task information.

```

>_MONGOSH
> use admin
< switched to db admin
> use taskdb
< switched to db taskdb
> db.createCollection("tasks")
< { ok: 1 }
taskdb> |

```

2. Insert Documents:

Insert a document for a task named "frontend development" assigned to "alice" with a status of "in progress" and a deadline of "2025-03-01".

```

> db.tasks.insertOne({name:"alice",task:"frontend development",status:"in progress",deadline:"2025-03-01"})
< {
  acknowledged: true,
  insertedId: ObjectId('67dc0e86990f9ff08c9059f4')
}

```

3. Update Documents:

a) Change the status of the "frontend development" task to "completed".

```

> db.task.updateOne({task:"frontend development"},{$set:{task:"completed"}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}

```

4.

5.

6. b) Extend the deadline of the "frontend development" task to "2025-04-01".

```

> db.task.updateMany({task:"frontend development"},{$set:{deadline:"2025-04-01"}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}

```

7.

c) Assign a new employee "bob" to the same task.

8.

9. Read Documents:

Retrieve all tasks that are assigned to "alice" and are marked as "in progress".

10. Delete Documents:

Delete the task document for "frontend development" from the `tasks` collection.

SCENARIO 2: TESTING TEAM MANAGEMENT

The company has a testing team responsible for verifying the functionality of different software products. Each tester is assigned to various test cases, and the status of each test case is tracked (e.g., "passed", "failed", "in progress").

1. Create Database and Collections:

Create a MongoDB database called `testdb`. Inside this database, create two collections: `testers` (for storing tester information) and `testcases` (for storing test case details).

```
> use testdb
< switched to db testdb
```

```
> show dbs
< Classrooms 200.00 KiB
  admin       40.00 KiB
  config      96.00 KiB
  librarydb   168.00 KiB
  local       80.00 KiB
  spacedb     72.00 KiB
  testdb      16.00 KiB
```

```
> db.createCollection("testers")
< { ok: 1 }
> db.createCollection("testcases")
< { ok: 1 }
```

```
> show collections
< testcases
  testers
```

2. Insert Documents:

Insert a document into the `testers` collection for a tester named "alice", with a role of "QA Engineer". Insert a test case document into the `testcases` collection, including the test case name "Login Test", status "in progress", assigned to "alice", and a priority of "high".

```
> db.testers.insertOne({eId:"e101",name:"Alice",roll:"QA Engineer"})
< {
  acknowledged: true,
  insertedId: ObjectId('67dbf183ca5096f1f917488c')
}
```

```
> db.testers.find()
< {
  _id: ObjectId('67dbf183ca5096f1f917488c'),
  eId: 'e101',
  name: 'Alice',
  roll: 'QA Engineer'
}
```

```
> db.testcases.insertOne({tcId:"tc101",tcName:"Login Test",status:"in progress",testerAssigned:"Alice",priority:"High"})
< {
  acknowledged: true,
  insertedId: ObjectId('67dbf245ca5096f1f917488e')
}
```

```
> db.testcases.find()
< {
  _id: ObjectId('67dbf245ca5096f1f917488e'),
  tcId: 'tc101',
  tcName: 'Login Test',
  status: 'passed',
  testerAssigned: 'Alice',
  priority: 'High'
}
```

3. Update Documents:

a) Change the status of "Login Test" to "passed".

```
> db.testcases.updateOne({tcName:"Login Test"},{$set:{status:"passed"}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

- b) Add a new test case called "Signup Test" with status "failed" and assign it to "bob".

```
> db.testcases.insertOne({tcId:"tc102",tcName:"Sign Up Test",status:"failed",testerAssigned:"Bob",priority:"High"})
< {
  acknowledged: true,
  insertedId: ObjectId('67dbf3deca5096f1f917488f')
}
```

- c) Update "alice's" role to "Senior QA Engineer".

```
> db.testers.updateOne({$and:[{eId:"e101"},{name:"Alice"}]},{$set:{roll:"Senior QA Engineer"}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

4. Read Documents:

Retrieve all test cases assigned to "alice" and list their names, statuses, and priorities.

```
> db.testcases.find({testerAssigned:"Alice"},{tcName:1,status:1,priority:1,_id:0})
< {
  tcName: 'Login Test',
  status: 'passed',
  priority: 'High'
}
```

5. Delete Documents:

Delete the "Signup Test" case from the `testcases` collection.

```
> db.testcases.deleteOne({tcName:"Sign Up Test"})
< {
  acknowledged: true,
  deletedCount: 1
}
```

```
> db.testcases.find()
< {
  _id: ObjectId('67dbf245ca5096f1f917488e'),
  tcId: 'tc101',
  tcName: 'Login Test',
  status: 'passed',
  testerAssigned: 'Alice',
  priority: 'High'
}
testdb> |
```

Scenario: Student Management System

The school needs to manage student records, which include the student's name, age, grade, and the courses they are enrolled in. The system allows CRUD (Create, Read, Update, Delete) operations to efficiently manage student data.

1. Create Database and Collection

Create a MongoDB database called `studentdb` to store student information. Inside this database, create a collection called `students`. This collection will contain the following fields for each student:

- `name`: The name of the student (string).
- `age`: The student's age (integer).
- `grade`: The grade the student is in (string, e.g., "10th").
- `courses`: An array of courses the student is enrolled in (array of strings).

2. Insert Document

- Insert a student named "Alice Johnson", who is 16 years old, in the 10th grade, and enrolled in "Math", "Science", and "English".
- Insert another student named "Bob Smith", who is 15 years old, in the 9th grade, and enrolled in "History", "Math", and "Geography"

3. Update Documents

- a) Change Alice Johnson's grade to "11th" after she advances to the next year.
- b) Add a new course, "History," to Alice Johnson's list of enrolled courses.
- c) Change Bob Smith's age to 16 after he celebrates his birthday.

4. Read Documents

- a) Retrieve all students who are enrolled in "Math".
- b) Retrieve all students who are in the 10th grade.
- c) Get all students who are older than 15 years of age.

5. Delete Documents

- a) Delete a specific student, such as "Bob Smith," from the **students** collection.
- b) Remove a course, like "Geography," from Bob Smith's list of courses.

SCENARIO-4

A hospital management system stores patient data in MongoDB.

The system needs to efficiently handle patient records, diagnoses, and treatment history while ensuring scalability and quick retrieval.

(a) Write MongoDB commands to create a new database named "HospitalDB" and a collection "patients".

```
> use HospitalDB
< switched to db HospitalDB
> db["patients"].find()
<

db.createCollection("patients")
{ ok: 1 }
```

b) Insert at least two sample patient documents with fields: patient_id, name, age, diagnosis, and treatment_plan.

```
db.patients.insertMany([{patient_id:"p1",name:"Alex",age:19,diagnosis:"LIVER",treatment_plan:""}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('67dbbb2e9e94f76e124f13b5')
  }
}
db.patients.insertMany([{patient_id:"p2",name:"Ale",age:19,diagnosis:"LIVER",treatment_plan:"done"},{patient_id:"p3",name:"Al",age:18,diagnosis:"DIABETES",treatment:"in progress"}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('67dbbcc99e94f76e124f13b6'),
    '1': ObjectId('67dbbcc99e94f76e124f13b7')
  }
}
```



```
db.patients.find()
{
  _id: ObjectId('67dbbb2e9e04f76e124f13b5'),
  patient_id: 'p1',
  name: 'Alex',
  age: 19,
  diagnosis: 'LIVER',
  treatment_plan: ''
}
{
  _id: ObjectId('67dbbcc99e04f76e124f13b6'),
  patient_id: 'p2',
  name: 'Ale',
  age: 19,
  diagnosis: 'LIVER',
  treatment_plan: 'done'
}
{
  _id: ObjectId('67dbbcc99e04f76e124f13b7'),
  patient_id: 'p3',
  name: 'Al',
  age: 18,
  diagnosis: 'DIABETES',
  treatment_plan: 'in progress'
}
```

c) Write a MongoDB query to retrieve all patients diagnosed with "Diabetes" from the "patients" collection. Modify the query to return only the name and treatment_plan fields while excluding _id.

```
> db.patients.find({diagnosis:"DIABETES"})
< {
  _id: ObjectId('67dbbcc99e04f76e124f13b7'),
  patient_id: 'p3',
  name: 'Al',
  age: 18,
  diagnosis: 'DIABETES',
  treatment_plan: 'in progress'
}
> db.patients.find({diagnosis:"DIABETES"},{name:1,treatment_plan:1,_id:0})
< {
  name: 'Al',
  treatment_plan: 'in progress'
}
```

d) Assume the hospital management decides to archive old patient records. Write a MongoDB query to drop the patient records.

```
> show collections
< patients
> db.patients.drop()
< true
```

RESULT:

CRUD Operations are performed in MongoDB successfully

Ex. Nos. 5	APPLYING FILTERING AND PROJECTIONS
Date: 16.02.2026	

Aim

To apply filtering and projection on a MongoDB collection using the find() method.

Code

Step 1: Create Collection and Insert Data

use HospitalDB

```
db.patients.insertMany([
  { patient_id: "p1", name: "Alex", age: 19, diagnosis: "LIVER", treatment_plan: "" },
  { patient_id: "p2", name: "Ale", age: 19, diagnosis: "LIVER", treatment_plan: "done" },
  { patient_id: "p3", name: "Al", age: 18, diagnosis: "DIABETES", treatment_plan: "in
progress" }
])
```

Step 2: Filtering (Select patients with Diabetes)

```
db.patients.find({ diagnosis: "DIABETES" })
```

Step 3: Projection (Display only name and treatment_plan)

```
db.patients.find(
  { diagnosis: "DIABETES" },
  { name: 1, treatment_plan: 1 }
)
```

Step 4: Excluding a Field (Hide _id field)

```
db.patients.find(
  { diagnosis: "DIABETES" },
  { name: 1, treatment_plan: 1, _id: 0 }
)
```

Step 5: Drop Collection (Archive patient records)

```
db.patients.drop()
```

Output

Filtering Output

```
[
  {
    "_id": ObjectId("..."),
    "patient_id": "p3",
    "name": "Al",
    "age": 18,
    "diagnosis": "DIABETES",
    "treatment_plan": "in progress"
  }
]
```

Projection Output (Including Fields)

```
[
  {
    "_id": ObjectId("..."),
    "name": "Al",
    "treatment_plan": "in progress"
  }
]
```

Projection Output (Excluding _id Field)

```
[
  {
    "name": "Al",
    "treatment_plan": "in progress"
  }
]
```

Result

Filtering and projection were successfully performed using MongoDB queries to retrieve specific documents and selected fields.

Ex. Nos. 6	PERFORM SCHEMA DESIGN AND INDEXING
Date: 23.03.2026	

Aim

To design a schema and apply indexing in MongoDB for efficient data storage and retrieval.

Code

Step 1: Create Collection and Insert Documents (Schema Design)

use HospitalDB

```
db.patients.insertMany([
  { patient_id: "p1", name: "Alex", age: 19, diagnosis: "LIVER", treatment_plan: "" },
  { patient_id: "p2", name: "Ale", age: 19, diagnosis: "LIVER", treatment_plan: "done" },
  { patient_id: "p3", name: "Al", age: 18, diagnosis: "DIABETES", treatment_plan: "in progress" }
])
```

Step 2: Create Index on Diagnosis Field

```
db.patients.createIndex({ diagnosis: 1 })
```

Step 3: Create Compound Index (diagnosis + age)

```
db.patients.createIndex({ diagnosis: 1, age: -1 })
```

Step 4: View Indexes

```
db.patients.getIndexes()
```

Step 5: Query Using Indexed Field

```
db.patients.find({ diagnosis: "DIABETES" })
```

Output

Inserted Documents

```
[
  { "_id": 1, "patient_id": "p1", "name": "Alex", "age": 19, "diagnosis": "LIVER",
    "treatment_plan": "" },
  { "_id": 2, "patient_id": "p2", "name": "Ale", "age": 19, "diagnosis": "LIVER",
    "treatment_plan": "done" },
```

```
{ "_id": 3, "patient_id": "p3", "name": "Al", "age": 18, "diagnosis": "DIABETES",  
  "treatment_plan": "in progress" }  
]
```

Indexes Created

```
[  
  { "key": { "_id": 1 }, "name": "_id_" },  
  { "key": { "diagnosis": 1 }, "name": "diagnosis_1" },  
  { "key": { "diagnosis": 1, "age": -1 }, "name": "diagnosis_1_age_-1" }  
]
```

Query Result

```
[  
  { "_id": 3, "patient_id": "p3", "name": "Al", "age": 18, "diagnosis": "DIABETES",  
    "treatment_plan": "in progress" }  
]
```

Result

A suitable schema was designed and indexes were created in MongoDB, improving query efficiency and data retrieval performance.

Ex. Nos. 7	BUILDING AGGREGATION PIPELINES
Date: 02.03.2026	

Aim

To build and execute aggregation pipelines in MongoDB for data processing and analysis.

Code

Step 1: Create Collection and Insert Data

use HospitalDB

```
db.patients.insertMany([
  { patient_id: "p1", name: "Alex", age: 19, diagnosis: "LIVER", treatment_plan: "ongoing", cost: 2000 },
  { patient_id: "p2", name: "Ale", age: 19, diagnosis: "LIVER", treatment_plan: "done", cost: 1500 },
  { patient_id: "p3", name: "Al", age: 18, diagnosis: "DIABETES", treatment_plan: "in progress", cost: 3000 },
  { patient_id: "p4", name: "John", age: 20, diagnosis: "DIABETES", treatment_plan: "ongoing", cost: 2500 }
])
```

Step 2: Aggregation Pipeline (Group by diagnosis and calculate average cost)

```
db.patients.aggregate([
  { $group: { _id: "$diagnosis", avgCost: { $avg: "$cost" } } }
])
```

Step 3: Aggregation with Match and Sort

```
db.patients.aggregate([
  { $match: { cost: { $gt: 2000 } } },
  { $group: { _id: "$diagnosis", totalCost: { $sum: "$cost" } } },
  { $sort: { totalCost: -1 } }
])
```

Output

Aggregation Output (Average Cost per Diagnosis)

```
[
  { "_id": "LIVER", "avgCost": 1750 },
  { "_id": "DIABETES", "avgCost": 2750 }
]
```

Aggregation Output (Filtered, Grouped, Sorted)

```
[  
  { "_id": "DIABETES", "totalCost": 5500 },  
  { "_id": "LIVER", "totalCost": 0 }  
]
```

Result

Aggregation pipelines were successfully built using stages like \$match, \$group, and \$sort to analyze and transform data in MongoDB.

Ex. Nos. 8	PERFORMING DATA TRANSFORMATION
Date: 09.03.2026	

Aim

To perform data transformation in MongoDB using aggregation pipeline operators.

Code

Step 1: Create Collection and Insert Data

use HospitalDB

```
db.patients.insertMany([
  { patient_id: "p1", name: "Alex", diagnosis: "liver", cost: 2000 },
  { patient_id: "p2", name: "Ale", diagnosis: "diabetes", cost: 1500 },
  { patient_id: "p3", name: "Al", diagnosis: "liver", cost: 3000 }
])
```

Step 2: Transform Data (Convert diagnosis to uppercase)

```
db.patients.aggregate([
  {
    $project: {
      name: 1,
      diagnosis: { $toUpper: "$diagnosis" },
      cost: 1
    }
  }
])
```

Step 3: Add New Field (Treatment Status based on cost)

```
db.patients.aggregate([
  {
    $project: {
      name: 1,
      cost: 1,
      status: {
        $cond: {
          if: { $gte: ["$cost", 2000] },
          then: "HIGH",
          else: "LOW"
        }
      }
    }
  }
])
```

```
}  
D)
```

Output

Transformed Diagnosis Field

```
[  
  { "_id": 1, "name": "Alex", "diagnosis": "LIVER", "cost": 2000 },  
  { "_id": 2, "name": "Ale", "diagnosis": "DIABETES", "cost": 1500 },  
  { "_id": 3, "name": "Al", "diagnosis": "LIVER", "cost": 3000 }  
]
```

Added Status Field

```
[  
  { "_id": 1, "name": "Alex", "cost": 2000, "status": "HIGH" },  
  { "_id": 2, "name": "Ale", "cost": 1500, "status": "LOW" },  
  { "_id": 3, "name": "Al", "cost": 3000, "status": "HIGH" }  
]
```

Result

Data transformation was successfully performed using aggregation operators like `$project`, `$toUpper`, and `$cond` to modify and enrich documents in MongoDB.

Ex. Nos. 9	ESTABLISHING CONNECTION BETWEEN MONGO DB AND JAVA
Date: 16.03.2026	

Aim

To establish connectivity between MongoDB and a Java application using MongoDB Java Driver.

Code

Step 1: Add MongoDB Driver Dependency (Maven)

```
<dependency>
  <groupId>org.mongodb</groupId>
  <artifactId>mongodb-driver-sync</artifactId>
  <version>4.11.1</version>
</dependency>
```

Step 2: Java Program to Connect and Insert Patient Data

```
import com.mongodb.client.MongoClient;
import com.mongodb.client.MongoClients;
import com.mongodb.client.MongoDatabase;
import com.mongodb.client.MongoCollection;
import org.bson.Document;

public class HospitalDBConnection {
    public static void main(String[] args) {

        // Create MongoDB client
        MongoClient mongoClient = MongoClients.create("mongodb://localhost:27017");

        // Access database
        MongoDatabase database = mongoClient.getDatabase("HospitalDB");

        // Access collection
        MongoCollection<Document> collection = database.getCollection("patients");

        // Create document
        Document doc = new Document("patient_id", "p1")
            .append("name", "Alex")
            .append("age", 19)
            .append("diagnosis", "DIABETES")
            .append("treatment_plan", "in progress");

        // Insert document
        collection.insertOne(doc);
```

```
System.out.println("Patient data inserted successfully");

// Close connection
mongoClient.close();
}
}
```

Output

Patient data inserted successfully

Result

MongoDB connectivity was successfully established with Java using the MongoDB driver, and data was inserted into the collection.

